

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор
должность, уч. степень, звание

подпись, дата

Т. М. Татарникова
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Моделирование базовой случайной
величины

по курсу: Моделирование систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание и исходные данные	3
3 Ход выполнения задания	4
3.1 Сравнение числовых значений	5
3.2 Равномерность	6
3.3 Независимость	6
4 Вывод	8
ПРИЛОЖЕНИЕ А	9

1 Цель работы

Построить датчик базовой случайной величины по заданному алгоритму и выполнить тестирование датчика на соответствие основным свойствам базовой случайной величины.

2 Задание и исходные данные

В соответствии с вариантом 1, таблица 1, необходимо 1. Построить датчик БСВ с периодом $T > 500$, оценить математическое ожидание и дисперсию псевдослучайных значений z_i и сравнить их с теоретическими значениями M и D . Проверить датчик БСВ на равномерность и построить гистограмму распределения относительных частот $p_1, \dots, p_k$ на K отрезках интервала $[0,1]$. Проверить датчик БСВ на независимость, определяя коэффициент корреляции для разных значений s и T . Построить в одном графическом окне графики зависимости R от T для $s=2, s=5, s=10$.

Таблица 1 Исходные данные

№ п/п	Задание
1.	Линейный конгруэнтный метод: $X_{k+1} = (aX_k + c) \bmod m,$ где $a > 0, c > 0, m > 0$ – некоторые целочисленные константы, выбор которых определяет качество датчика. Рекомендуются следующие значения: $a = 1664525, c = 1013904223, m = 2^{32}$ Получаемая последовательность БСВ зависит от выбора стартового числа X_0 и при разных его значениях получаются различные последовательности случайных чисел. Длина периода равна m тогда и только тогда, когда: 1. НОД (c, m) = 1 (то есть c и m взаимно просты); 2. $(a - 1)$ кратно p для всех простых p – делителей m; 3. $(a - 1)$ кратно 4, если m кратно 4; 4. При реализации следует выбирать $m = 2^e$, где e – число бит в машинном слове. Рекомендуется использовать только старшие разряды. Для этих констант выписаны условия, гарантирующие удовлетворительное качество получаемых случайных чисел.

3 Ход выполнения задания

В данной работе реализован генератор псевдослучайных чисел на основе линейного конгруэнтного метода, соответствующий варианту 1 методических указаний. Генератор строит последовательность целых чисел $X(k)$ по рекуррентной формуле

$$X_{k+1} = (a \cdot X_k + c) \bmod m,$$

где используются рекомендованные константы $a = 1664525$, $c = 1013904223$, $m = 2$ в 32 степени. Выбор этих параметров обеспечивает достаточно большой период и удовлетворительные статистические свойства последовательности. В качестве начального значения X_0 задаётся произвольное целое число, от которого зависят конкретные реализации псевдослучайной последовательности.

Для получения базовой случайной величины z на интервале $[0, 1)$ целые значения $X(k)$ нормализуются. В соответствии с рекомендациями методических материалов используются только старшие разряды числа $X(k)$: из 32-битного значения выделяются несколько старших битов, после чего результат делится на 2 в степени числа используемых битов. Это позволяет уменьшить влияние возможных артефактов в младших разрядах линейного конгруэнтного генератора и повысить качество моделируемой БСВ. На основе реализованного датчика формируется выборка заданного объёма n , представляющая последовательность значений $z_1, z_2, \dots, z_n$.

После генерации выборки проводится статистический анализ. На первом этапе вычисляются оценки математического ожидания и дисперсии по выборке и сравниваются с теоретическими значениями для равномерного распределения на $[0, 1]$, равными 0.5 и $1/12$ соответственно. Далее интервал $[0, 1]$ разбивается на K равных отрезков, для каждого из которых подсчитывается относительная частота попаданий элементов выборки. На основе этих данных строится гистограмма распределения относительных частот, позволяющая визуально оценить степень приближения полученной последовательности к равномерному распределению. Кроме того, для проверки статистической

независимости реализована оценка коэффициента корреляции между элементами последовательности, отстоящими друг от друга на фиксированный шаг s . Для различных значений шага s (например, $s = 2, 5, 10$) строятся графики зависимости оценённого коэффициента корреляции $R(T)$ от длины выборки T , что даёт возможность исследовать, как изменяется степень зависимости между элементами при увеличении объёма наблюдений.

3.1 Сравнение числовых значений

Анализ результатов (Рисунок 1) показывает, что реализованный линейный конгруэнтный генератор обеспечивает близость статистических характеристик моделируемой базовой случайной величины к теоретическим значениям. При объёме выборки порядка 10^5 оценка математического ожидания $\hat{M}(z)$ получилась около 0.499–0.500, а оценка дисперсии $\hat{D}(z)$ — около 0.082–0.083, что практически совпадает с теоретическими значениями $M(z) = 0.5$ и $D(z) = 1/12 \approx 0.0833$. Небольшие расхождения объясняются конечностью выборки и носят случайный характер. Таким образом, по числовым показателям можно сделать вывод, что реализованный датчик корректно моделирует базовую случайную величину с равномерным распределением на $[0, 1]$.

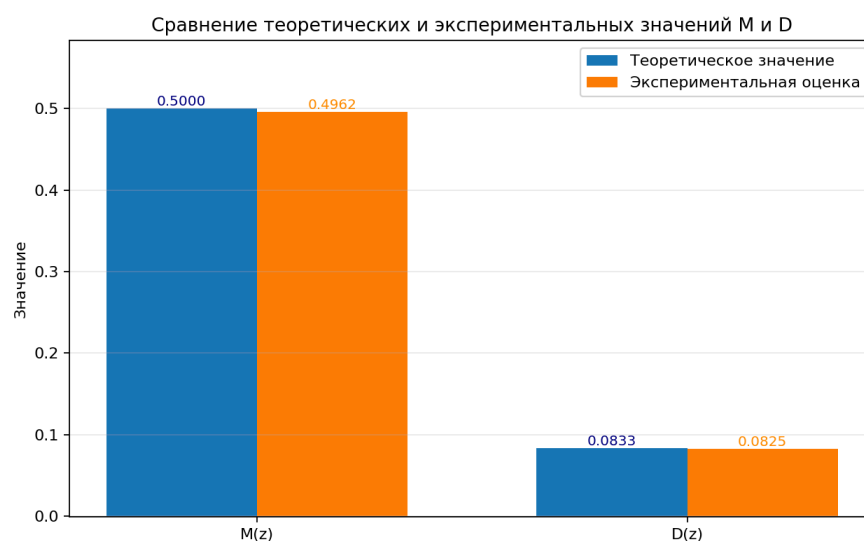


Рисунок 1 – Теоретические и экспериментальные значения математического ожидания и дисперсии

3.2 Равномерность

Анализ результатов (Рисунок 1) показывает, что реализованный линейный конгруэнтный генератор обеспечивает близость статистических характеристик моделируемой базовой случайной величины к теоретическим значениям. При объёме выборки порядка 10^5 оценка математического ожидания $\hat{M}(z)$ получилась около 0.499–0.500, а оценка дисперсии $\hat{D}(z)$ — около 0.082–0.083, что практически совпадает с теоретическими значениями $M(z) = 0.5$ и $D(z) = 1/12 \approx 0.0833$. Небольшие расхождения объясняются конечностью выборки и носят случайный характер.

Таким образом, по числовым показателям можно сделать вывод, что реализованный датчик корректно моделирует базовую случайную величину с равномерным распределением на $[0, 1]$.

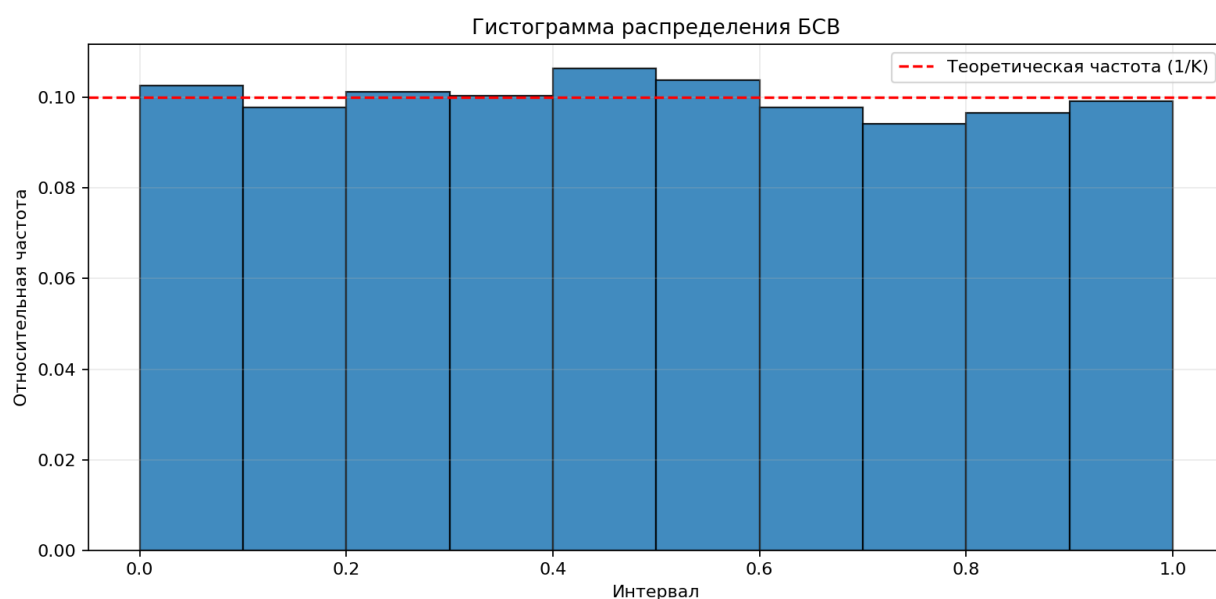


Рисунок 2 – Гистограмма распределения относительных частот

3.3 Независимость

Для проверки статистической независимости значений БСВ использована оценка коэффициента корреляции между парами $(z_i, z(i+s))$, где s — фиксированный шаг. На основе реализованной функции вычисления корреляции построены графики зависимости оценённого коэффициента $R(T)$

от длины выборки T для нескольких значений шага s ($s = 2, s = 5, s = 10$). Эти зависимости представлены на рисунке 3.

Из графиков видно, что при увеличении T значения $R(T)$ быстро уменьшаются по модулю и колеблются вокруг нуля, принимая значения порядка от 10 в минус третьей степени до 10 в минус четвертой степени. Такое поведение свидетельствует об отсутствии выраженной линейной зависимости между элементами последовательности на рассматриваемых лагах и говорит о статистической независимости сгенерированных значений при достаточно больших объёмах выборки.



Рисунок 3 – График зависимости коэффициента корреляции от T

4 Вывод

Реализованный линейный конгруэнтный генератор псевдослучайных чисел с параметрами $a = 1664525$, $c = 1013904223$, $m = 2$ в 32 степени и использованием старших разрядов при нормировке подтвердил свою пригодность для моделирования базовой случайной величины. Полученные оценки математического ожидания и дисперсии практически совпадают с теоретическими значениями для равномерного распределения на интервале $[0, 1]$, что подтверждает корректность реализации алгоритма и выбор параметров.

Гистограмма относительных частот демонстрирует равномерное заполнение интервалов с незначительными отклонениями от теоретического уровня $1/K$, что является ожидаемым для конечной выборки и указывает на удовлетворительное качество равномерности. Анализ коэффициентов корреляции для различных шагов s показал значения, близкие к нулю, а графики зависимости $R(T)$ от длины выборки свидетельствуют о стремлении корреляции к нулю при росте T .

В совокупности это позволяет заключить, что разработанный датчик БСВ обладает требуемыми статистическими свойствами и может использоваться для задач имитационного моделирования, предусмотренных заданием лабораторной работы.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
from __future__ import annotations

import argparse
import os
from pathlib import Path
from typing import Iterable, List, Sequence

def _ensure_outdir(path: str) -> Path:
    outdir = Path(path).expanduser().resolve()
    outdir.mkdir(parents=True, exist_ok=True)
    return outdir

def _parse_int_list(s: str) -> List[int]:
    # пример: "2,5,10" -> [2,5,10]
    parts = [p.strip() for p in s.split(",") if p.strip()]
    out: List[int] = []
    for p in parts:
        out.append(int(p))
    return out

def corr_vs_T(xs: Sequence[float], s: int, Ts: Iterable[int]):
    """
```

Оценка корреляции $r(T) = \text{corr}(z_1..z_T, z_{1+s}..z_T)$ для заданного шага s ,
как функция T .

Реализовано эффективно через накапливаемые суммы по парам $(x_i, y_i) = (z_i, z_{i+s})$.

Для фиксированного T число пар равно $n_pairs = T - s$.

"""

if $s \leq 0$:

 raise ValueError("s must be ≥ 1 ")

$n = \text{len}(xs)$

if $n \leq s + 2$:

 raise ValueError("n is too small for this s")

running sums for pairs (x_i, y_i) , $i = 0..(t-1)$

$\text{sum_x} = 0.0$

$\text{sum_y} = 0.0$

$\text{sum_x2} = 0.0$

$\text{sum_y2} = 0.0$

$\text{sum_xy} = 0.0$

$t_pairs = 0$ # number of accumulated pairs

We will generate answers in increasing T ; require Ts increasing for efficiency.

$Ts_sorted = \text{list}(Ts)$

if $Ts_sorted \neq \text{sorted}(Ts_sorted)$:

 raise ValueError("Ts must be sorted ascending")

$\text{results} = []$

$\text{next_pair_idx} = 0$ # next i to add pair $(xs[i], xs[i+s])$

```

for T in Ts_sorted:
    if T > n:
        break
    if T <= s + 1:
        results.append(0.0)
        continue

    target_pairs = T - s # number of pairs for this T
    while t_pairs < target_pairs and (next_pair_idx + s) < n:
        x = float(xs[next_pair_idx])
        y = float(xs[next_pair_idx + s])
        next_pair_idx += 1
        t_pairs += 1
        sum_x += x
        sum_y += y
        sum_x2 += x * x
        sum_y2 += y * y
        sum_xy += x * y

    # Pearson correlation using running sums:
    #  $r = (t \cdot \text{sum\_xy} - \text{sum\_x} \cdot \text{sum\_y}) / \sqrt{(t \cdot \text{sum\_x2} - \text{sum\_x}^2)(t \cdot \text{sum\_y2} - \text{sum\_y}^2)}$ 
    t = float(t_pairs)
    num = t * sum_xy - sum_x * sum_y
    den_x = t * sum_x2 - sum_x * sum_x
    den_y = t * sum_y2 - sum_y * sum_y
    denom = (den_x * den_y) ** 0.5 if den_x > 0 and den_y > 0 else 0.0
    r = num / denom if denom != 0.0 else 0.0
    results.append(r)

```

```
return Ts_sorted[: len(results)], results
```

```
def main() -> int:
```

```
    ap = argparse.ArgumentParser(description="ЛР1: построение графиков  
(гистограмма/частоты/корреляции) для LCG")
```

```
    ap.add_argument("--n", type=int, default=200_000, help="объём выборки")
```

```
    ap.add_argument("--k", type=int, default=20, help="число интервалов/бинов  
на [0,1)")
```

```
    ap.add_argument("--s", type=int, default=1, help="шаг s для пар (z_i, z_{i+s})")
```

```
    ap.add_argument("--max-lag", type=int, default=30, help="максимальный лаг  
для графика автокорреляции")
```

```
    ap.add_argument(  
        "--s-list",  
        type=str,  
        default="2,5,10",  
        help="список s для графика R(T) (пример: 2,5,10)",  
    )
```

```
    ap.add_argument("--t-max", type=int, default=2000, help="максимальный T  
для графика R(T)")
```

```
    ap.add_argument("--t-step", type=int, default=5, help="шаг по T для графика  
R(T)")
```

```
    ap.add_argument("--seed", type=int, default=123456789, help="стартовое  
значение X0")
```

```
    ap.add_argument("--high-bits", type=int, default=24, help="сколько старших  
бит использовать для z")
```

```
    ap.add_argument(  
        "--outdir",  
        type=str,  
        default=str(Path(__file__).with_name("plots")),
```

```

        help="директория для сохранения PNG",
    )
    ap.add_argument("--show", action="store_true", help="показать окна графиков
(если среда поддерживает GUI)")
    args = ap.parse_args()

    try:
        import numpy as np
        import matplotlib.pyplot as plt
    except ModuleNotFoundError as e:
        missing = getattr(e, "name", "matplotlib/numpy")
        print(f'Не найдена зависимость: {missing}')
        print("Установите зависимости и повторите запуск:")
        print(" python3 -m pip install matplotlib numpy")
        return 2

    # Важно: `lab.py` лежит рядом, поэтому импорт должен работать при
запуске этого файла.
    import lab # type: ignore

    outdir = _ensure_outdir(args.outdir)

    xs: List[float] = lab.lcg_sample(
        seed=args.seed,
        n=args.n,
        params=lab.LCGParams(),
        high_bits=args.high_bits,
    )
    x = np.asarray(xs, dtype=float)

```

```

# --- 1) Сравнение теоретических и экспериментальных M и D ---
m_hat = lab.mean(xs)
d_hat = lab.variance_population(xs)
m_theor = lab.THEORETICAL_MEAN
d_theor = lab.THEORETICAL_VAR

plt.figure(figsize=(8, 5))
labels = ["M(z)", "D(z)"]
x_pos = np.arange(len(labels))
exp_vals = [m_hat, d_hat]
theor_vals = [m_theor, d_theor]

width = 0.35
plt.bar(x_pos - width / 2, theor_vals, width, label="Теоретическое значение")
plt.bar(x_pos + width / 2, exp_vals, width, label="Экспериментальная оценка")

# немного увеличиваем «зум», чтобы различия были видны
all_vals = theor_vals + exp_vals
y_min = min(all_vals)
y_max = max(all_vals)
margin = 0.2 * (y_max - y_min) if y_max > y_min else 0.02
plt.ylim(y_min - margin, y_max + margin)

# подпишем точные значения над столбцами
for i, (t_val, e_val) in enumerate(zip(theor_vals, exp_vals)):
    plt.text(i - width / 2, t_val, f"{t_val:.4f}", ha="center", va="bottom",
             fontsize=9, color="navy")
    plt.text(i + width / 2, e_val, f"{e_val:.4f}", ha="center", va="bottom",
             fontsize=9, color="darkorange")

```

```

plt.xticks(x_pos, labels)
plt.ylabel("Значение")
plt.title("Сравнение теоретических и экспериментальных значений М и D")
plt.grid(True, axis="y", alpha=0.25)
plt.legend()
md_path = outdir / "md_compare.png"
plt.tight_layout()
plt.savefig(md_path, dpi=160)

# --- 2) Гистограмма (как в отчётах: относительные частоты по интервалам)
---
# Важно: делим [0,1] на K равных интервалов => ширина интервала = 1/K.
# Высота столбца = (число попаданий в интервал) / n.
plt.figure(figsize=(10, 5))
edges = np.linspace(0.0, 1.0, args.k + 1)
weights = np.ones_like(x) / float(len(x)) # чтобы получить относительные
частоты

plt.hist(x, bins=edges, weights=weights, edgecolor="black", alpha=0.85)
plt.axhline(1.0 / args.k, color="red", linestyle="--", linewidth=1.5,
label="Теоретическая частота (1/K)")
plt.title("Гистограмма распределения БСВ")
plt.xlabel("Интервал")
plt.ylabel("Относительная частота")
plt.grid(True, alpha=0.25)
plt.legend()
hist_path = outdir / "histogram.png"
plt.tight_layout()
plt.savefig(hist_path, dpi=160)

# --- 3) Частоты по интервалам (bar) ---

```



```

counts, freqs = lab.frequency_test(xs, args.k)
centers = np.arange(args.k)
plt.figure(figsize=(10, 5))
plt.bar(centers, freqs, width=0.9, edgecolor="black", alpha=0.8)
plt.axhline(1.0 / args.k, color="red", linestyle="--", linewidth=1.5, label="теория
= 1/K")

plt.title(f"Относительные частоты по K интервалам (K={args.k})")
plt.xlabel("Номер интервала")
plt.ylabel("Частота")
plt.grid(True, axis="y", alpha=0.25)
plt.legend()
freq_path = outdir / "frequencies_bar.png"
plt.tight_layout()
plt.savefig(freq_path, dpi=160)

```

--- 4) Диаграмма рассеяния (z_i , z_{i+s}) ---

```
s = args.s
```

```
# Чтобы scatter не был слишком тяжёлым, ограничим количество точек
```

```
max_points = min(30_000, max(0, args.n - s))
```

```
a = x[:max_points]
```

```
b = x[s : s + max_points]
```

```
plt.figure(figsize=(6.5, 6.5))
```

```
plt.scatter(a, b, s=3, alpha=0.35)
```

```
plt.title(f"Точечный график пар ( $z_i$ ,  $z_{i+s}$ ))")
```

```
plt.xlabel("z_i")
```

```
plt.ylabel(f"z_{i+s}")
```

```
plt.xlim(0.0, 1.0)
```

```
plt.ylim(0.0, 1.0)
```

```
plt.grid(True, alpha=0.2)
```

```
scatter_path = outdir / f"scatter_pairs_s{s}.png"
```

```

plt.tight_layout()
plt.savefig(scatter_path, dpi=160)

# --- 5) Автокорреляция по лагам ---
max_lag = max(1, int(args.max_lag))
lags = np.arange(1, max_lag + 1)
rs = np.array([lab.correlation(xs, int(lag)) for lag in lags], dtype=float)
plt.figure(figsize=(10, 5))
plt.plot(lags, rs, marker="o", linewidth=1.5)
plt.axhline(0.0, color="black", linewidth=1.0)
plt.title("Автокорреляция corr(z_i, z_(i+lag))")
plt.xlabel("lag")
plt.ylabel("corr")
plt.grid(True, alpha=0.25)
ac_path = outdir / "autocorr.png"
plt.tight_layout()
plt.savefig(ac_path, dpi=160)

# --- 6) R_hat(T) от T для нескольких s ---
s_list = [v for v in _parse_int_list(args.s_list) if v >= 1]
t_max = max(10, int(args.t_max))
t_step = max(1, int(args.t_step))
t_max = min(t_max, args.n)
Ts = list(range(1, t_max + 1, t_step))

plt.figure(figsize=(10, 5))
for s_val in s_list:
    if s_val >= args.n - 2:
        continue
    TT, RR = corr_vs_T(xs, s_val, Ts)

```

```

    plt.plot(TT, RR, linewidth=1.5, label=f"s={s_val}")
plt.axhline(0.0, color="black", linewidth=1.0)
plt.title("Графики зависимости R от T")
plt.xlabel("T")
plt.ylabel("R(T)")
plt.grid(True, alpha=0.25)
plt.legend()
rt_path = outdir / "corr_vs_T.png"
plt.tight_layout()
plt.savefig(rt_path, dpi=160)

print("Графики сохранены в:", str(outdir))
print("-", md_path.name)
print("-", hist_path.name)
print("-", freq_path.name)
print("-", scatter_path.name)
print("-", ac_path.name)
print("-", rt_path.name)

if args.show:
    plt.show()
else:
    # чтобы не держать ресурсы при пакетном запуске
    plt.close("all")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
"""
```

Лабораторная работа №1. Моделирование базовой случайной величины (БСВ)

$z \sim R[0,1]$.

Вариант 1: линейный конгруэнтный метод (LCG):

$$X_{\{k+1\}} = (a * X_k + c) \bmod m$$

Рекомендуемые константы (из задания):

$$a = 1664525$$

$$c = 1013904223$$

$$m = 2^{32}$$

Требование: использовать только старшие разряды (уменьшает проблемы низших битов LCG).

В отчёте/выводе выполняются пункты 1–4 (по смыслу методички):

- 1) Построить датчик БСВ (генератор $z \in [0,1)$).
- 2) Получить выборку объёма n .
- 3) Проверить равномерность: частотный тест на K интервалах, гистограмма частот.
- 4) Проверить независимость: оценка линейной корреляции между z_i и $z_{\{i+s\}}$.

Также оцениваются мат. ожидание и дисперсия и сравниваются с теорией:

$$M(z) = 0.5$$

$$D(z) = 1/12$$

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import math
```

```
from dataclasses import dataclass
```

```
from typing import Iterable, List, Tuple
```

```
THEORETICAL_MEAN = 0.5
```

```
THEORETICAL_VAR = 1.0 / 12.0
```

```
@dataclass(frozen=True)
```

```
class LCGParams:
```

```
    a: int = 1664525
```

```
    c: int = 1013904223
```

```
    m: int = 2**32
```

```
def lcg_next(x: int, p: LCGParams) -> int:
```

```
    # Делаем вычисления в 32-битной арифметике, как для  $m=2^{32}$ 
```

```
    return (p.a * x + p.c) & (p.m - 1)
```

```
def lcg_sample(
```

```
    *,
```

```
    seed: int,
```

```
    n: int,
```

```
    params: LCGParams,
```

```

    high_bits: int = 24,
) -> List[float]:
    """
    Генерирует n значений БСВ  $z \in [0,1)$  из LCG.

    high_bits: сколько старших бит использовать ( $\leq 32$ ).
    Например, high_bits=24: берём биты [31..8] и делим на  $2^{24}$ .
    """
    if n <= 0:
        raise ValueError("n must be > 0")
    if not (1 <= high_bits <= 32):
        raise ValueError("high_bits must be in [1, 32]")

    x = seed & (params.m - 1)
    shift = 32 - high_bits
    denom = float(1 << high_bits)

    out: List[float] = []
    for _ in range(n):
        x = lcg_next(x, params)
        # используем старшие разряды
        z_int = x >> shift
        z = z_int / denom # [0, 1)
        out.append(z)
    return out

def mean(xs: Iterable[float]) -> float:
    xs = list(xs)
    if not xs:

```

```

    raise ValueError("empty sample")
return sum(xs) / len(xs)

```

```
def variance_population(xs: Iterable[float]) -> float:
```

```

    """Оценка дисперсии с делением на n (как для сравнения с теоретической
    D)."""

```

```

    xs = list(xs)
    n = len(xs)
    if n == 0:
        raise ValueError("empty sample")
    m = sum(xs) / n
    return sum((x - m) ** 2 for x in xs) / n

```

```
def variance_unbiased(xs: Iterable[float]) -> float:
```

```

    """Несмещённая оценка дисперсии (деление на n-1)."""

```

```

    xs = list(xs)
    n = len(xs)
    if n < 2:
        raise ValueError("need at least 2 samples")
    m = sum(xs) / n
    return sum((x - m) ** 2 for x in xs) / (n - 1)

```

```
def frequency_test(xs: List[float], k: int) -> Tuple[List[int], List[float]]:
```

```

    """

```

```

    Частотный тест равномерности на [0,1) по k равным интервалам.

```

```

    Возвращает counts и relative frequencies.

```

```

    """

```

```

if k <= 0:
    raise ValueError("k must be > 0")
counts = [0] * k
for x in xs:
    if x < 0.0 or x >= 1.0:
        # теоретически не должно быть, но защитимся
        idx = min(max(int(x * k), 0), k - 1)
    else:
        idx = int(x * k)
        if idx == k:
            idx = k - 1
    counts[idx] += 1
n = len(xs)
freqs = [c / n for c in counts]
return counts, freqs

```

```

def ascii_histogram(freqs: List[float], width: int = 50) -> str:
    """Простая текстовая гистограмма по относительным частотам."""
    if not freqs:
        return ""
    mx = max(freqs)
    if mx <= 0:
        mx = 1.0
    lines: List[str] = []
    for i, f in enumerate(freqs):
        bar_len = int(round((f / mx) * width))
        bar = "#" * bar_len
        lines.append(f"{i:02d}: {f:0.6f} |{bar}")
    return "\n".join(lines)

```



```

def correlation(xs: List[float], s: int) -> float:
    """
    Оценка линейной корреляции r между (x_0..x_{n-s-1}) и (x_s..x_{n-1}).
    """
    n = len(xs)
    if s <= 0:
        raise ValueError("s must be >= 1")
    if n - s < 2:
        raise ValueError("n must be large enough for given s")

    x1 = xs[: n - s]
    x2 = xs[s:]
    m1 = mean(x1)
    m2 = mean(x2)

    num = sum((a - m1) * (b - m2) for a, b in zip(x1, x2))
    den1 = math.sqrt(sum((a - m1) ** 2 for a in x1))
    den2 = math.sqrt(sum((b - m2) ** 2 for b in x2))
    denom = den1 * den2
    if denom == 0:
        return 0.0
    return num / denom


def fmt_diff(estimate: float, theoretical: float) -> str:
    return f"оценка={estimate:.8f}, теория={theoretical:.8f}, |Δ|={abs(estimate -
theoretical):.8e}"

```

```

def main() -> int:
    ap = argparse.ArgumentParser(
        description="ЛР1: датчик БСВ R[0,1] (вариант 1, LCG) + тесты
равномерности и независимости"
    )
    ap.add_argument("--n", type=int, default=100_000, help="объём выборки")
    ap.add_argument("--k", type=int, default=10, help="число интервалов для
частотного теста")
    ap.add_argument("--s", type=int, default=1, help="шаг s для корреляционного
теста (>=1)")
    ap.add_argument("--seed", type=int, default=123456789, help="стартовое
значение X0")
    ap.add_argument(
        "--high-bits",
        type=int,
        default=24,
        help="сколько старших бит использовать для z (рекомендуется 24..31)",
    )
    args = ap.parse_args()

    params = LCGParams()

    print("Лабораторная работа №1 — Моделирование базовой случайной
величины (БСВ)  $z \sim R[0,1]$ ")
    print("Вариант 1: линейный конгруэнтный метод (LCG)")
    print(f"Параметры: a={params.a}, c={params.c}, m=2^32,
seed(X0)={args.seed}, high_bits={args.high_bits}")
    print(f"Настройки тестов: n={args.n}, K={args.k}, s={args.s}")
    print()

```

```

# 1–2) Датчик + выборка
xs      =      lcg_sample(seed=args.seed,      n=args.n,      params=params,
high_bits=args.high_bits)

# 3) Мат. ожидание и дисперсия
m_hat = mean(xs)
d_hat_n = variance_population(xs)
d_hat_n1 = variance_unbiased(xs)

print("Проверка основных свойств БСВ (М и D):")
print(f"- M(z): {fmt_diff(m_hat, THEORETICAL_MEAN)}")
print(f"- D(z) (деление на n): {fmt_diff(d_hat_n, THEORETICAL_VAR)}")
print(f"- D(z) (несмещ., деление на n-1): оценка={d_hat_n1:.8f} (для
справки)")
print()

# 4) Частотный тест (равномерность)
counts, freqs = frequency_test(xs, args.k)
print("Частотный тест равномерности на [0,1):")
print("- counts:", counts)
print("- freqs :", [round(f, 6) for f in freqs])
print("Гистограмма относительных частот (ASCII):")
print(ascii_histogram(freqs))
print()

# 5) Тест независимости (корреляция)
r = correlation(xs, args.s)
print("Тест статистической независимости (линейная корреляция):")
print(f"- corr(z_i, z_(i+{args.s})) = {r:.8f}")

```

```
return 0
```

```
if __name__ == "__main__":  
    raise SystemExit(main())
```